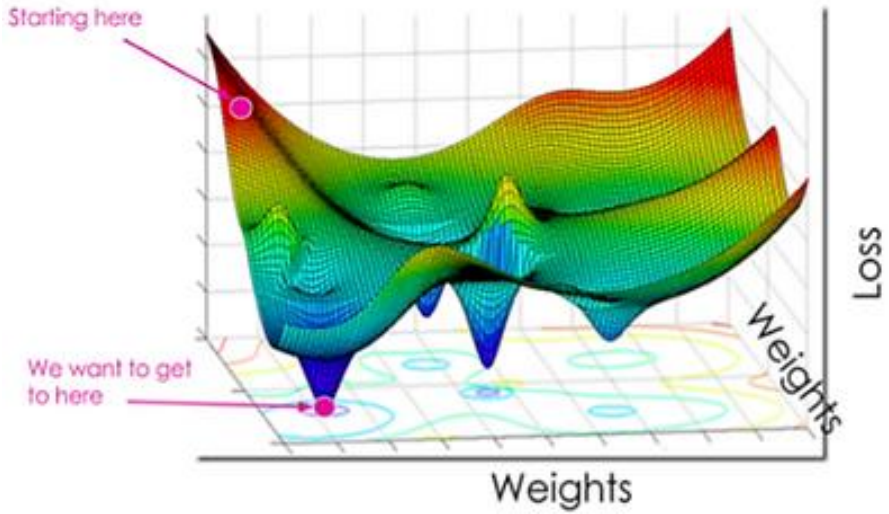
Introducción a Aprendizaje Profundo

Deep Learning

Funciones de pérdida



Funciones de pérdida

Las más comunes son las ya dadas.

Para clasificación, categorical_crossentropy o binary_crossentropy según si el problema es binario o multiclase.

Para regresión, MSE, o también Huber-Loss, que es una combinación de MSE y MAE (para que sea menos sensible a outliers).

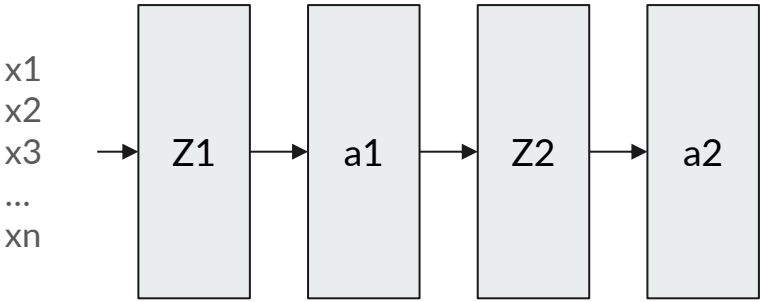
Aunque se pueden utilizar muchas más - vemos en la documentación

Funciones de activación

Funciones de activación

- Ninguna
- Escalón (step)
- Sigmoidea (y softmax)
- Tangente hiperbólica
- ReLU
- Leaky ReLU

Si no uso función de activación...



$$\vec{Z}_i^{(1)} = \vec{W}^{(1)} \cdot \vec{X}_i + \vec{b}^{(1)}$$

$$\vec{Z}_i^{(2)} = \vec{W}^{(2)} \cdot \vec{Z}_i^{(1)} + \vec{b}^{(2)} = \vec{W}^{(2)} \cdot \left(\vec{W}^{(1)} \cdot \vec{X}_i + \vec{b}^{(1)} \right) + \vec{b}^{(2)}$$

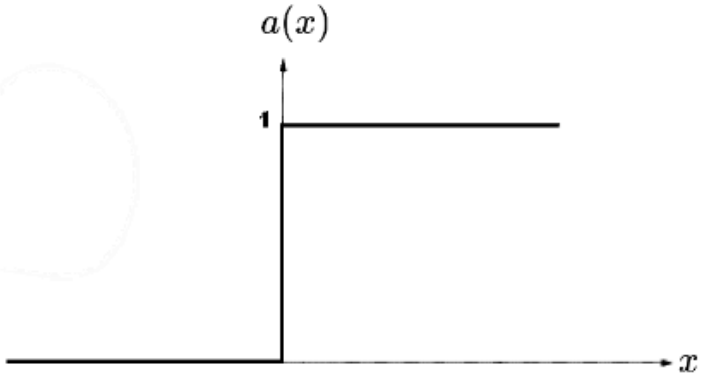
$$= \left(\vec{W}^{(2)} \cdot \vec{W}^{(1)} \right) \cdot \vec{X}_i + \left(\vec{W}^{(2)} \cdot \vec{b}^{(1)} + \vec{b}^{(2)} \right)$$

$$\vec{W}' \qquad \vec{b}'$$

$$\hat{y}_i = \vec{W}' \cdot \vec{X}_i + \vec{b}'$$

es lo mismo que hacer una regresión lineal múltiple.

Función escalón



$$a(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases} \longrightarrow a'(x) = \begin{cases} 0, & x > 0 \\ 0, & x < 0 \\ ? & x = 0 \end{cases}$$

$$\partial L/\partial W_{1,1}^{(1)} = \partial L/\partial \hat{y}_i \cdot \partial \hat{y}_i/\partial a_{i,1}^{(1)} \cdot \partial a_{i,1}^{(1)}/\partial Z_{i,1}^{(1)} \cdot \partial Z_{i,1}^{(1)}/\partial W_{1,1}^{(1)}$$

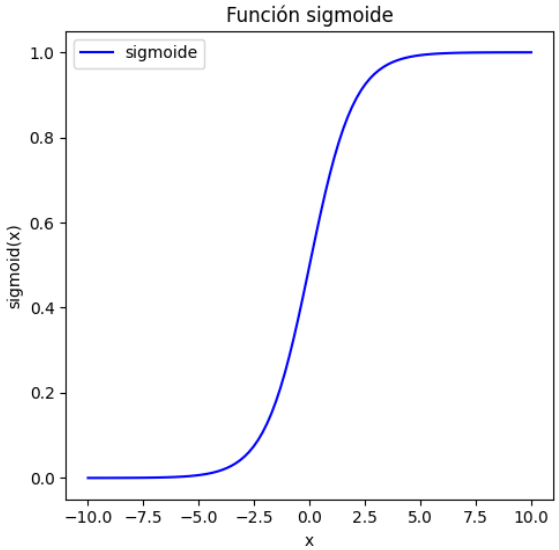
$$\partial L/\partial W_{1,1}^{(1)} = 0$$

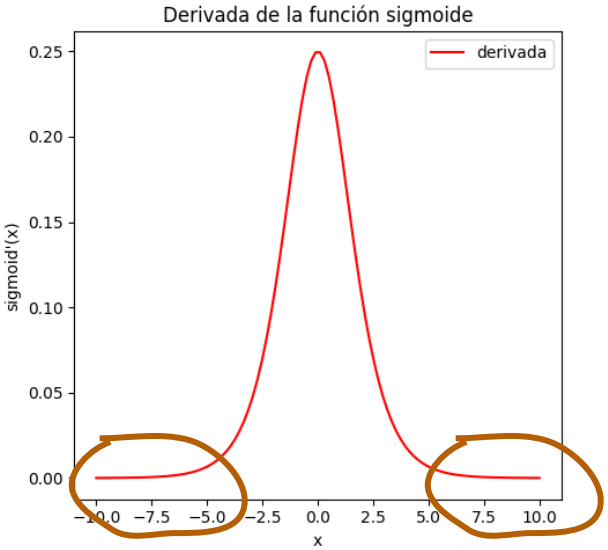
$$W_{1,1}^{(1)} \leftarrow W_{1,1}^{(1)} - \alpha \cdot \cancel{\partial L/\partial W_{1,1}^{(1)}}$$

se obtienen gradientes nulos y corta el entrenamiento

Función sigmoide

$$\partial L/\partial W_{1,1}^{(1)} = \partial L/\partial \hat{y}_i \cdot \partial \hat{y}_i/\partial a_{i,1}^{(1)} \cdot \partial a_{i,1}^{(1)}/\partial Z_{i,1}^{(1)} \cdot \partial Z_{i,1}^{(1)}/\partial W_{1,1}^{(1)}$$





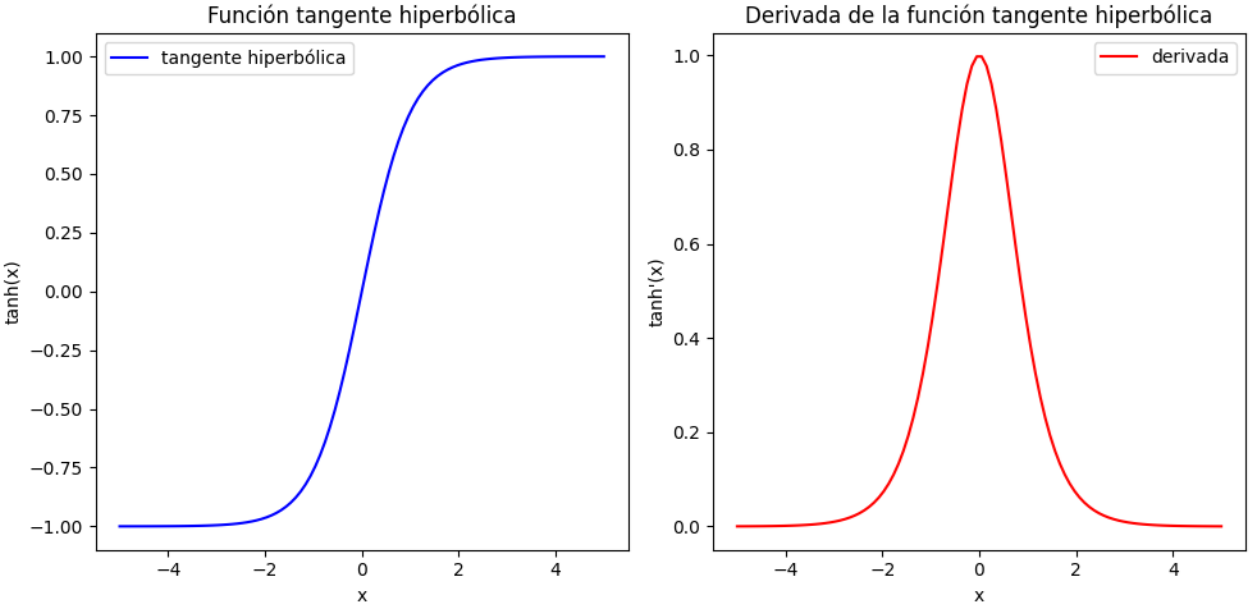
Se puede dar una optimización, ya que los gradientes no son nulos.

Sin embargo existen intervalos (prácticamente todo el eje x) en los cuales los gradientes son muy bajos respecto del valor absoluto de x. Esto se conoce como “vanishing gradients” o gradientes que se desvanecen.

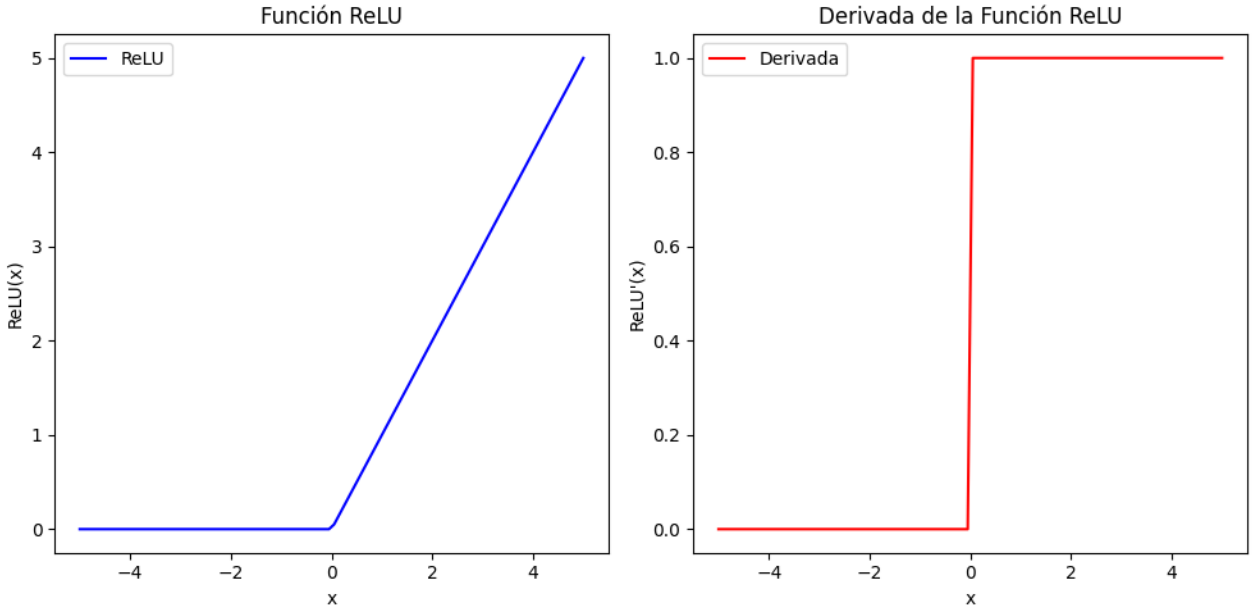
El modelo aprende, pero de forma lenta.

$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Tangente hiperbólica



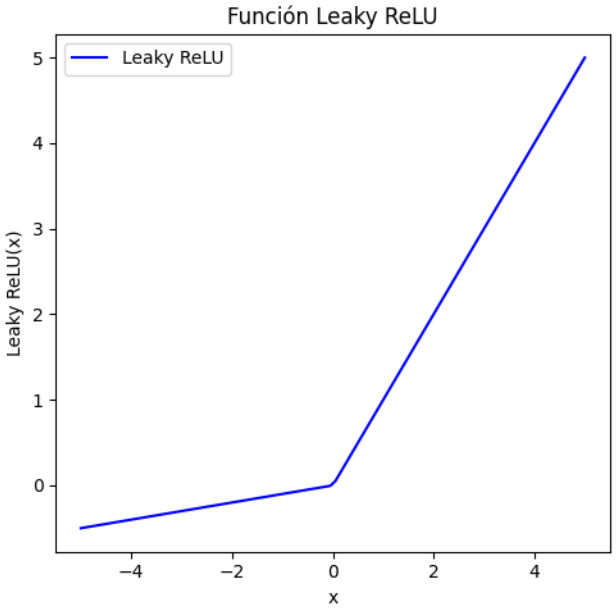
ReLU (Rectified Linear Unit)

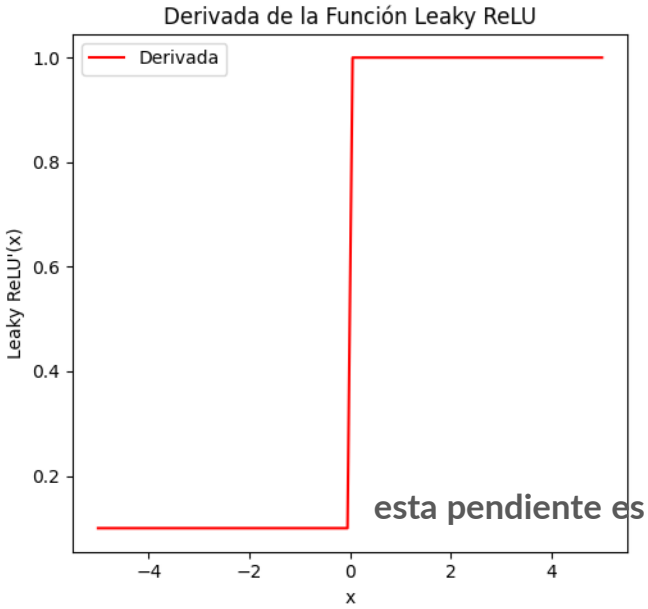


Además de que simplifica mucho las cuentas a la hora de hacer los pasos backward o forward, el entrenamiento resulta muy rápido (pendiente alta)

Sin embargo, para entradas negativas, el gradiente queda nulo, y se puede perder poder de entrenamiento.

Leaky ReLU





esta pendiente es usualmente 0.01

Leaky ReLU

- Resuelve el problema de las neuronas “muertas” de la ReLU al permitir seguir entrenando el modelo con valores negativos.
- El pequeño flujo de información que deja pasar con entradas negativas sirve para continuar el entrenamiento, aunque de forma más lenta.

Regularización

En caso de sufrir overfitting...

Ya vimos que en estos casos la regularización puede servir.

Una opción puede ser bajar la complejidad del modelo, cambiando la arquitectura.

Sin embargo puede que no haya una solución lo suficientemente óptima o entrenar una gran cantidad de arquitecturas distintas sea muy time-consuming.

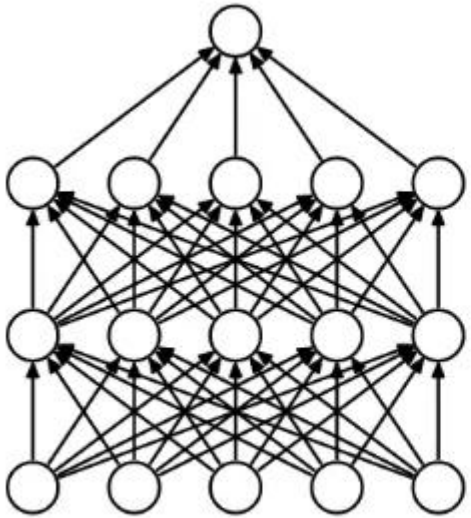
¿Cómo podemos regularizar?

Regularización

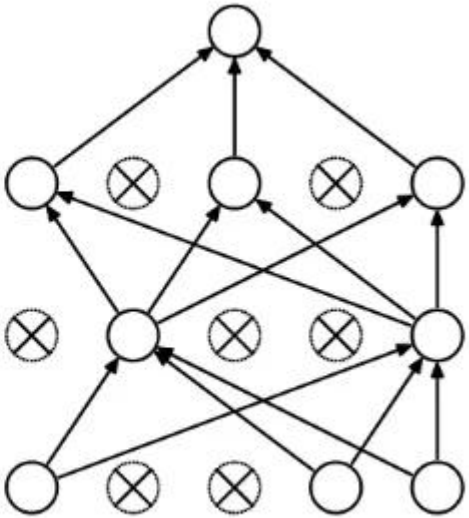
- Se puede utilizar las regularizaciones L1 y L2, esto es agregar a la función de costo un término ponderado de las normas de los parámetros de la red.

Dropout

A cada peso se le asigna una determinada **probabilidad p** (fija por el usuario) de ser ***nulo***. De esta forma se eliminan conexiones que puede que hayan **sobre-aprendido**, agregando robustez.



(a) Standard Neural Net



(b) After applying dropout.

Ajuste de hiperparámetros

Ajuste de hiperparámetros (HP tuning)

- Ajustar hiperparámetros de forma manual, con grid search o random search ahora es más costoso.
- El enfoque bayesiano es más adecuado.